# 🚀 Collections Framework Masterclass

# Chapter 3 — List Interface (Complete Deep Dive)

> **Interview Importance:** ⭐⭐⭐⭐⭐

The **List** interface is one of the most frequently used collection types in Java. Almost every Java application uses `ArrayList` or `LinkedList`, so interviewers expect you to know the List interface thoroughly.

---

# What is List?

A **List** is an **ordered collection** that allows:

- Duplicate elements
- Null elements (implementation dependent)
- Positional access (index-based)
- Preserves insertion order

Package:

```
java.util.List
```

Declaration:

```
public interface List<E> extends Collection<E>
```

Hierarchy:

```
                Iterable
                    ↑
               Collection
                    ↑
                  List
          ____________|_____________
         |            |             |
    ArrayList    LinkedList     Vector
                                   |
                                 Stack
```

---

# Characteristics of List

## 1. Ordered Collection

Elements are stored in insertion order.

```
List<String> list = new ArrayList<>();

list.add("Java");
list.add("Python");
list.add("C++");

System.out.println(list);
```

Output

```
[Java, Python, C++]
```

The order remains exactly the same.

---

## 2. Allows Duplicates

Unlike Set, List allows duplicate values.

```
List<Integer> list = new ArrayList<>();

list.add(10);
list.add(10);
list.add(20);

System.out.println(list);
```

Output

```
[10, 10, 20]
```

---

## 3. Allows Null Values

Most List implementations allow multiple null values.

```
List<String> list = new ArrayList<>();

list.add(null);
list.add(null);

System.out.println(list);
```

Output

```
[null, null]
```

---

## 4. Index-Based Access

Every element has an index.

```
Index : 0   1   2
Value : A   B   C
```

Retrieve element

```
list.get(1);
```

Output

```
B
```

---

# Why List?

Suppose you're developing an online shopping application.

Cart Items

```
1. Laptop
2. Mouse
3. Keyboard
```

Order matters.

Duplicates are allowed.

You need index-based access.

Perfect use case for List.

---

# List Interface Methods

Apart from Collection methods, List introduces many new methods.

---

# add(int index, E element)

Adds element at a specific position.

```
List<String> list = new ArrayList<>();

list.add("A");
list.add("C");

list.add(1, "B");

System.out.println(list);
```

Output

```
[A, B, C]
```

---

# get()

Returns element at index.

```
String value = list.get(2);
```

Output

```
C
```

Time Complexity

ArrayList

```
O(1)
```

LinkedList

```
O(n)
```

Interview Favorite ⭐⭐⭐⭐⭐

---

# set()

Replaces element.

```
list.set(1, "Java");

System.out.println(list);
```

Output

```
[A, Java, C]
```

Returns old value.

---

# remove(int index)

Removes element at index.

```
list.remove(1);
```

Output

```
[A, C]
```

---

# indexOf()

Returns first occurrence.

```
List<Integer> list =
Arrays.asList(10,20,30,20);

System.out.println(
list.indexOf(20)
);
```

Output

```
1
```

---

# lastIndexOf()

Returns last occurrence.

```
System.out.println(
list.lastIndexOf(20)
);
```

Output

```
3
```

---

# subList()

Returns a view of the original list.

```
List<Integer> list =
Arrays.asList(10,20,30,40,50);

System.out.println(
list.subList(1,4)
);
```

Output

```
[20,30,40]
```

Interview Favorite ⭐⭐⭐⭐⭐

Important:

- Start index → Inclusive
- End index → Exclusive

---

# listIterator()

Returns

```
ListIterator<E>
```

Unlike Iterator

ListIterator can

- Move forward
- Move backward
- Update elements
- Add elements
- Remove elements

Example

```
ListIterator<String> itr =
list.listIterator();

while(itr.hasNext())
{
    System.out.println(itr.next());
}
```

---

# Iterator vs ListIterator

| Feature | Iterator | ListIterator |
|---|---|---|
| Forward | Yes | Yes |
| Backward | No | Yes |
| Add | No | Yes |
| Set | No | Yes |
| Remove | Yes | Yes |
| Only List | No | Yes |

Interview Favorite ⭐⭐⭐⭐⭐

---

# Array vs List

| Array | List |
|---|---|
| Fixed Size | Dynamic |
| Primitive Support | Objects Only |
| Faster | Flexible |
| No Built-in Methods | Rich API |
| Cannot Resize | Automatically Resizes |

---

# List Implementations

Java provides four major implementations.

```
ArrayList

LinkedList

Vector

Stack
```

Each has different characteristics.

We'll study each in separate chapters.

---

# ArrayList

Internally

```
Resizable Array
```

Best for

- Random Access
- Read-heavy operations

---

# LinkedList

Internally

```
Doubly Linked List
```

Best for

- Frequent Insertions
- Frequent Deletions

---

# Vector

Internally

```
Resizable Array
```

Difference

Methods are synchronized.

Thread-safe.

Slower than ArrayList.

---

# Stack

Extends Vector.

Follows

```
LIFO
```

Operations

```
push()

pop()

peek()
```

Legacy class.

Nowadays

Prefer

```
Deque
```

---

# Equality in List

Example

```
List<Integer> l1 =
Arrays.asList(1,2,3);

List<Integer> l2 =
Arrays.asList(1,2,3);

System.out.println(
l1.equals(l2)
);
```

Output

```
true
```

Lists compare

- Order
- Elements

If order changes

```
[1,2,3]

[2,1,3]
```

Result

```
false
```

Interview Favorite

---

# hashCode() in List

Contract

```
Equal Lists

↓
```

```
Equal HashCodes
```

Hash code depends on

- Order
- Elements

---

# Immutable List

Java 9 introduced

```
List<String> list =
List.of(
"A",
"B",
"C"
);
```

Cannot modify.

```
list.add("D");
```

Throws

```
UnsupportedOperationException
```

Interview Favorite ⭐⭐⭐⭐

---

# Arrays.asList() vs List.of()

## Arrays.asList()

```
List<String> list =
Arrays.asList("A","B");
```

- Fixed-size
- Allows set()

- Backed by original array
- Allows null values

---

### List.of()

```
List<String> list =
List.of("A","B");
```

- Completely immutable
- Does not allow add()
- Does not allow remove()
- Does not allow set()
- Does **not** allow null values

---

# Arrays.asList() Example

```
List<String> list =
Arrays.asList("A","B");

list.set(0,"Java");

System.out.println(list);
```

Output

```
[Java,B]
```

Works.

But

```
list.add("C");
```

Throws exception.

---

# List.of() Example

```
List<String> list =
List.of("A","B");

list.set(0,"Java");
```

Throws

```
UnsupportedOperationException
```

---

# Time Complexity

| Operation | ArrayList | LinkedList |
|---|---|---|
| add(end) | O(1)* | O(1) |
| add(index) | O(n) | O(n) |
| get(index) | O(1) | O(n) |
| remove(index) | O(n) | O(n) |
| contains() | O(n) | O(n) |
| iteration | O(n) | O(n) |

*Amortized O(1)

---

# Common Mistakes

## Mistake 1

```
list.get(100);
```

Throws

```
IndexOutOfBoundsException
```

### Mistake 2

```
List<String> list =
List.of("A");

list.add("B");
```

Exception

---

### Mistake 3

```
Arrays.asList(1,2,3)
      .add(4);
```

Exception

---

## Real-world Use Cases

### Student List

```
List<Student> students;
```

---

### Shopping Cart

```
List<Product> cart;
```

---

### Search Results

```
List<Employee> employees;
```

---

### API Response

```
List<UserDTO> users;
```

---

### Database Records

```
List<Order> orders;
```

---

# Frequently Asked Interview Questions

#### Q1. Difference between List and Set?

| List | Set |
|---|---|
| Allows duplicates | No duplicates |
| Ordered | Generally unordered (except LinkedHashSet/TreeSet) |
| Index-based | No index |

---

#### Q2. Which List implementation is fastest?

Depends.

- Reading → ArrayList
- Insertion/Deletion (middle, frequent structural changes) → LinkedList
- Thread-safe → Vector

---

#### Q3. Why does List allow duplicates?

Because List is designed to preserve insertion order and sequence. Duplicate values are considered valid elements.

---

### Q4. Difference between Arrays.asList() and List.of()?

| Arrays.asList() | List.of() |
|---|---|
| Fixed-size | Immutable |
| Allows set() | No set() |
| Allows null | No null |
| Backed by array | Independent immutable collection |

---

### Q5. Difference between Iterator and ListIterator?

`ListIterator` supports bidirectional traversal and modification (`add()`, `set()`), while `Iterator` supports only forward traversal and element removal.

---

### Q6. What happens if you access an invalid index?

An `IndexOutOfBoundsException` is thrown.

---

### Q7. Why is ArrayList generally preferred over LinkedList?

Because most applications perform far more reads than insertions/deletions in the middle. `ArrayList` provides O(1) random access, is cache-friendly due to contiguous memory, and usually offers better overall performance.

---

# Key Takeaways

- `List` extends the `Collection` interface.
- Lists preserve insertion order.
- Lists allow duplicate and (implementation-dependent) null elements.
- Lists provide index-based operations such as `get()`, `set()`, `add(index)`, and `remove(index)`.
- `ListIterator` is more powerful than `Iterator`.
- `Arrays.asList()` returns a fixed-size list, while `List.of()` returns an immutable list.
- Choose `ArrayList` for fast random access and `LinkedList` when frequent insertions/deletions are required.

---

# Next Chapter

## 🚀 Chapter 4 – ArrayList (Complete Internal Working)

We'll cover:

- Internal Architecture
- Dynamic Array Concept
- Capacity vs Size
- Default Capacity
- Growth Algorithm (1.5x Expansion)
- Source Code Walkthrough
- `ensureCapacity()`
- `trimToSize()`
- Amortized Time Complexity
- Memory Layout
- Performance Analysis
- Interview Questions (15+)
- Real-world Scenarios